

Material de apoio — Diagramas UML e modelos C4 com Mermaid

Atividade prática: criação de diagramas técnicos com IA

1. Objetivo da atividade

Nesta atividade, você irá usar uma ferramenta de IA para gerar diagramas técnicos a partir de um estudo de caso simples.

O objetivo não é apenas gerar diagramas visualmente bonitos. O objetivo principal é aprender a transformar requisitos de um sistema em representações técnicas que ajudem a explicar sua estrutura, seus atores, seus fluxos e suas integrações.

Nesta atividade, serão trabalhados quatro tipos de diagramas:

Modelo C4

- Diagrama C4 de Contexto
- Diagrama C4 de Containers

UML

- Diagrama de Caso de Uso
- Diagrama de Sequência

A diferença é que agora os diagramas serão gerados usando **Mermaid**, uma linguagem textual para criação de diagramas.

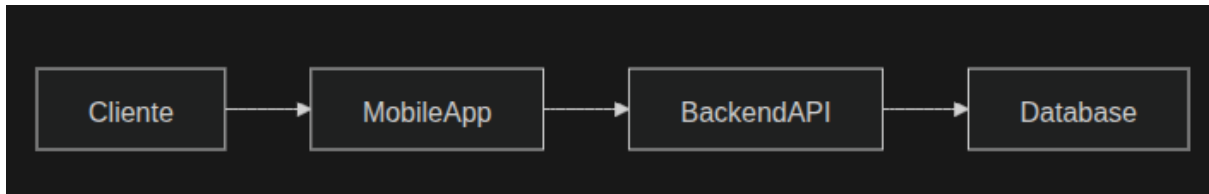
2. O que é Mermaid?

Mermaid é uma linguagem que permite criar diagramas a partir de texto.

Em vez de desenhar manualmente caixas, setas e conexões, você escreve uma estrutura textual, e a ferramenta transforma esse texto em um diagrama visual.

Exemplo:

```
flowchart LR
  Cliente --> MobileApp
  MobileApp --> BackendAPI
  BackendAPI --> Database
```



Esse código representa um fluxo simples em que o Cliente usa o Mobile App, o Mobile App chama o Backend API, e o Backend API acessa o Database.

3. Por que usar Mermaid?

O Mermaid é útil porque permite criar diagramas de forma rápida, simples e versionável.

Em projetos de software, isso é importante porque os diagramas podem ser armazenados junto com a documentação do projeto, por exemplo em arquivos `README.md`, `ARCHITECTURE.md` ou documentos dentro da pasta `docs/`.

Além disso, o Mermaid permite que a IA gere rapidamente uma primeira versão do diagrama, que depois deve ser revisada pela equipe.

A IA ajuda a acelerar a criação, mas a responsabilidade de verificar se o diagrama faz sentido continua sendo dos desenvolvedores.

4. Estudo de caso: Shop4u

O **Shop4u** é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

O cliente usa o aplicativo para visualizar produtos, buscar produtos, ver detalhes de um produto, receber recomendações personalizadas, adicionar produtos ao carrinho e finalizar compras.

O sistema também utiliza serviços externos para processar pagamentos e enviar notificações de confirmação.

5. Requisitos principais

O sistema deve permitir que o cliente:

- visualize produtos;
- busque produtos;
- veja detalhes de um produto;
- receba recomendações personalizadas;
- adicione produtos ao carrinho;
- finalize uma compra;
- receba confirmação do pedido;
- acompanhe o pedido.

Esses requisitos serão usados como base para a construção dos diagramas.

6. Elementos principais do sistema

Os diagramas devem considerar os seguintes elementos:

Elemento	Descrição
Cliente	Usuário que acessa o aplicativo e realiza compras
Mobile App	Interface mobile usada pelo cliente
Backend API	Camada responsável por organizar regras de negócio e integrar os demais serviços
Database	Banco de dados responsável por armazenar usuários, produtos, carrinhos, pedidos e histórico
Serviço de IA	Serviço usado para recomendar produtos com base no histórico do cliente
Gateway de Pagamento	Serviço externo responsável por processar pagamentos
Serviço de Notificações	Serviço externo responsável por enviar confirmações e alertas ao cliente

7. Relações principais entre os elementos

Use as relações abaixo como base para construir os diagramas:

- Cliente usa o Mobile App;
- Mobile App chama o Backend API;
- Backend API acessa o Database;

- Backend API consulta o Serviço de IA;
- Backend API chama o Gateway de Pagamento;
- Backend API chama o Serviço de Notificações.

A ideia principal é que o cliente interage com o aplicativo, o aplicativo chama o backend, e o backend coordena as integrações com banco de dados, IA, pagamento e notificações.

8. Fluxo principal de compra

O fluxo principal esperado é:

1. Cliente acessa o aplicativo.
2. Cliente visualiza ou busca produtos.
3. Cliente seleciona um produto.
4. Cliente adiciona o produto ao carrinho.
5. Cliente clica em finalizar compra.
6. Mobile App envia a solicitação para o Backend API.
7. Backend API valida os dados no Database.
8. Backend API chama o Gateway de Pagamento.
9. Gateway de Pagamento retorna pagamento aprovado.
10. Backend API salva o pedido no Database.
11. Backend API chama o Serviço de Notificações.
12. Mobile App exibe a confirmação da compra ao Cliente.

Esse fluxo será usado principalmente no Diagrama UML de Sequência.

9. Como estruturar prompts para gerar diagramas em Mermaid

Para gerar um bom diagrama com IA, o prompt precisa informar cinco pontos principais.

9.1 Tipo de diagrama

Informe claramente o tipo de diagrama que deve ser criado.

Exemplos:

- Diagrama C4 de Contexto
- Diagrama C4 de Containers
- Diagrama UML de Caso de Uso
- Diagrama UML de Sequência

9.2 Formato de saída

Informe que a saída deve ser em Mermaid.

Isso é importante porque a IA pode responder com texto explicativo, uma imagem conceitual ou uma descrição do diagrama. Neste exercício, o objetivo é gerar código Mermaid.

9.3 Contexto do sistema

Explique brevemente o que o sistema faz.

Exemplo:

O Shop4u é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

9.4 Elementos que devem aparecer

Liste os elementos principais que devem ser representados.

Exemplo:

- Cliente
- Mobile App
- Backend API
- Database
- Serviço de IA
- Gateway de Pagamento
- Serviço de Notificações

9.5 Relações ou fluxo

Explique como os elementos se conectam ou qual sequência deve ser representada.

Exemplo:

- Cliente usa o Mobile App;
 - Mobile App chama o Backend API;
 - Backend API acessa o Database;
 - Backend API chama o Serviço de IA;
 - Backend API chama o Gateway de Pagamento;
 - Backend API chama o Serviço de Notificações.
-

10. Estrutura geral de prompt

Use este modelo quando quiser criar seu próprio prompt:

Gere um **[TIPO DO DIAGRAMA]** em Mermaid para o sistema Shop4u.

Contexto:

O Shop4u é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

Inclua:

[Liste os elementos que devem aparecer no diagrama]

Relações ou fluxo:

[Liste as relações entre os elementos ou as etapas do fluxo]

Regras:

- Use sintaxe Mermaid compatível
- Use nomes curtos nos blocos
- Mantenha o diagrama simples e legível
- Não detalhe implementação interna
- Retorne apenas o código Mermaid

Esse modelo ajuda a criar prompts claros sem ficarem longos demais.

11. Diagrama C4 de Contexto

11.1 O que este diagrama deve mostrar

O Diagrama C4 de Contexto mostra o sistema principal em relação ao ambiente externo.

Neste nível, o Shop4u deve aparecer como uma única caixa. Ainda não é necessário mostrar Mobile App, Backend API ou Database.

A pergunta principal desse diagrama é:

Quem usa o sistema e quais sistemas externos se conectam a ele?

Esse diagrama é útil para apresentar o sistema de forma ampla, sem entrar em detalhes técnicos internos.

11.2 Prompt recomendado

Gere um Diagrama C4 de Contexto em Mermaid para o sistema Shop4u.

Contexto:

O Shop4u é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

Inclua:

- Cliente
- Shop4u
- Serviço de IA
- Gateway de Pagamento
- Serviço de Notificações

Relações:

- Cliente usa o Shop4u
- Shop4u consulta o Serviço de IA para recomendar produtos
- Shop4u usa o Gateway de Pagamento para processar pagamentos
- Shop4u usa o Serviço de Notificações para enviar confirmações de pedido

Regras:

- Use C4Context
- Mantenha o Shop4u como uma única caixa
- Não detalhe Mobile App, Backend API ou Database
- Use nomes curtos nas relações
- Retorne apenas o código Mermaid

11.3 Exemplo esperado em Mermaid

flowchart LR

cliente[Cliente]

shop4u[Shop4u
Sistema de e-commerce mobile]

ia[Serviço de IA
Recomenda produtos]

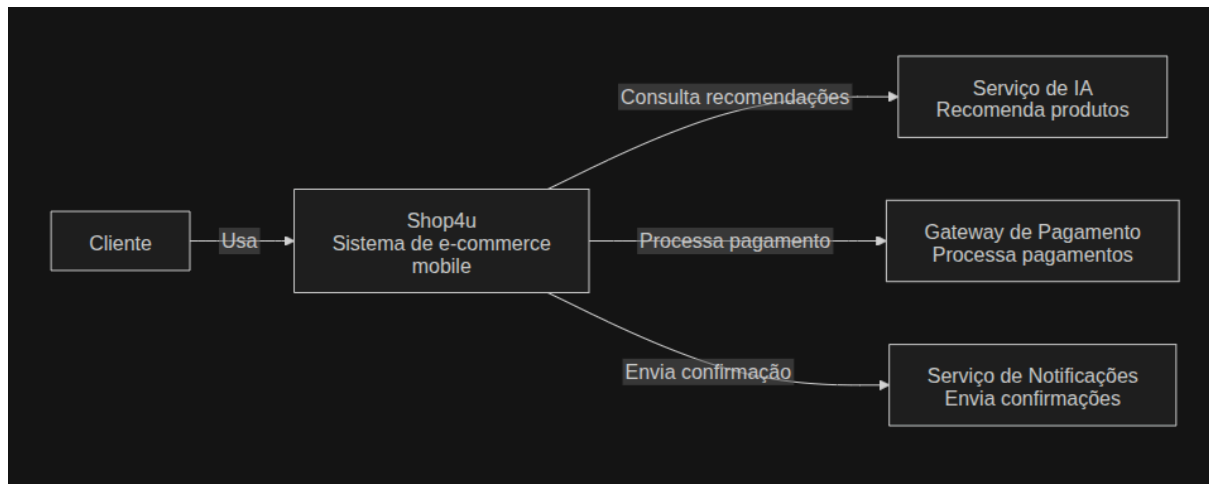
pagamento[Gateway de Pagamento
Processa pagamentos]

notificacoes[Serviço de Notificações
Envia confirmações]

cliente -->|Usa| shop4u

shop4u -->|Consulta recomendações| ia

shop4u -->|Processa pagamento| pagamento
shop4u -->|Envia confirmação| notificacoes



11.4 Como revisar este diagrama

Verifique se:

- o Shop4u aparece como sistema principal;
- o Cliente aparece fora do sistema;
- os serviços externos aparecem separados do Shop4u;
- Mobile App, Backend API e Database não aparecem neste nível;
- as relações estão simples e compreensíveis.

12. Diagrama C4 de Containers

12.1 O que este diagrama deve mostrar

O Diagrama C4 de Containers mostra os principais blocos técnicos do sistema.

Neste nível, o Shop4u não aparece mais apenas como uma única caixa. Ele é dividido em partes maiores, como Mobile App, Backend API e Database.

A pergunta principal desse diagrama é:

Quais são os grandes blocos técnicos da solução e como eles se comunicam?

Esse diagrama ainda não deve detalhar classes, métodos, controllers, services ou repositories.

12.2 Prompt recomendado

Gere uma representação do Diagrama C4 de Containers em Mermaid usando flowchart LR para o sistema Shop4u.

Contexto:

O Shop4u é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

Inclua:

- Cliente
- Mobile App
- Backend API
- Database
- Serviço de IA
- Gateway de Pagamento
- Serviço de Notificações

Relações:

- Cliente usa o Mobile App
- Mobile App chama o Backend API
- Backend API lê e grava dados no Database
- Backend API consulta o Serviço de IA para recomendações
- Backend API chama o Gateway de Pagamento para processar pagamentos
- Backend API chama o Serviço de Notificações para enviar confirmações

Regras:

- Use flowchart LR
- Não use C4Container, C4Context, Person, Container, ContainerDb, System_Ext ou Rel
- Coloque Mobile App, Backend API e Database dentro de uma subgraph chamada Shop4u
- Mantenha Serviço de IA, Gateway de Pagamento e Serviço de Notificações fora da subgraph
- Não inclua controllers, services, repositories, classes ou métodos
- Retorne apenas o código Mermaid

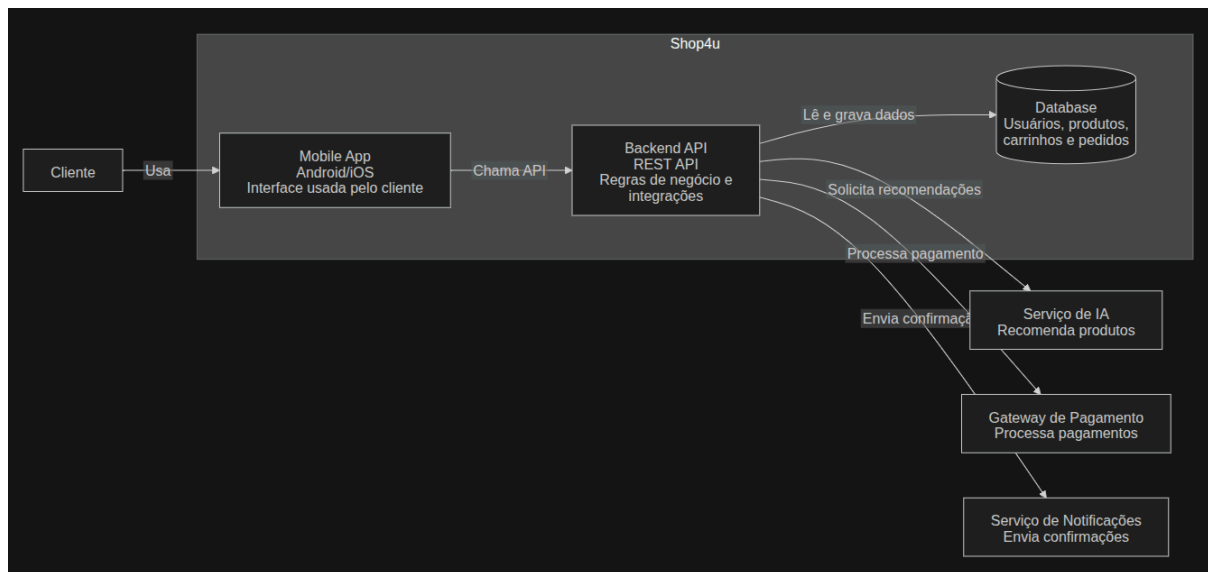
12.3 Exemplo esperado em Mermaid

```
flowchart LR
  cliente[Cliente]

  subgraph shop4u[Shop4u]
    app["Mobile App<br/>Android/iOS<br/>Interface usada pelo cliente"]
    api["Backend API<br/>REST API<br/>Regras de negócio e integrações"]
    db["Database<br/>Usuários, produtos, carrinhos e pedidos"]
  end
```

ia[Serviço de IA
Recomenda produtos]
pagamento[Gateway de Pagamento
Processa pagamentos]
notificacoes[Serviço de Notificações
Envia confirmações]

cliente -->|Usa| app
app -->|Chama API| api
api -->|Lê e grava dados| db
api -->|Solicita recomendações| ia
api -->|Processa pagamento| pagamento
api -->|Envia confirmação| notificacoes



12.4 Como revisar este diagrama

Verifique se:

- o Cliente está fora da fronteira do sistema;
- Mobile App, Backend API e Database estão dentro do Shop4u;
- o Backend API aparece como elemento central de comunicação;
- o Database é acessado pelo Backend API;
- os serviços externos estão fora do sistema principal;
- o diagrama não mostra detalhes internos de código.

13. Diagrama UML de Caso de Uso

13.1 O que este diagrama deve mostrar

O Diagrama de Caso de Uso mostra quais funcionalidades o ator pode acessar no sistema.

Neste caso, o ator principal é o **Cliente**.

A pergunta principal desse diagrama é:

O que o cliente pode fazer no Shop4u?

Esse diagrama não mostra fluxo de telas, backend, banco de dados ou código. Ele mostra as funcionalidades do sistema do ponto de vista do usuário.

No Mermaid, vamos representar esse diagrama usando **flowchart**, porque o Mermaid não possui uma sintaxe nativa completa para Diagrama de Caso de Uso UML.

Nesta adaptação:

- o Cliente será representado como um bloco simples à esquerda;
- os casos de uso ficarão dentro da fronteira do sistema Shop4u;
- os serviços externos ficarão à direita;
- as conexões indicam quais funcionalidades o Cliente acessa e quais serviços externos são usados.

13.2 Prompt recomendado

Gere um Diagrama UML de Caso de Uso em Mermaid usando flowchart LR para o sistema Shop4u.

Contexto:

O Shop4u é um aplicativo mobile de e-commerce com recomendação de produtos por IA.

Ator principal:

- Cliente

Casos de uso:

- Fazer login
- Visualizar produtos
- Buscar produtos
- Ver detalhes do produto
- Receber recomendações
- Adicionar produto ao carrinho
- Finalizar compra
- Acompanhar pedido

Sistemas externos:

- Serviço de IA
- Gateway de Pagamento
- Serviço de Notificações

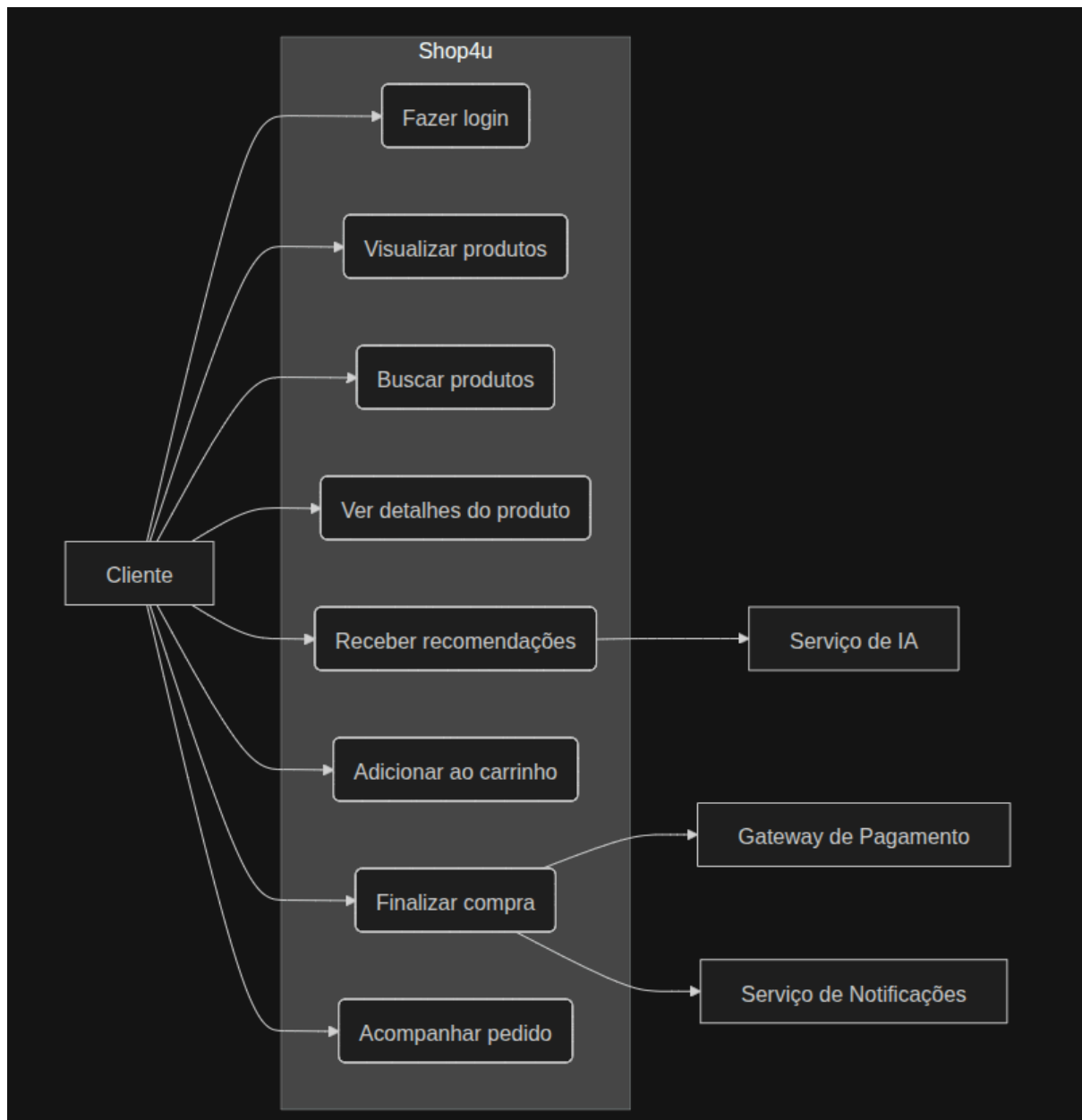
Relações:

- Cliente acessa os casos de uso do Shop4u
- Receber recomendações usa o Serviço de IA
- Finalizar compra usa o Gateway de Pagamento
- Finalizar compra aciona o Serviço de Notificações

Regras:

- Use flowchart LR
- Represente o Cliente como um bloco simples à esquerda
- Coloque os casos de uso dentro de uma subgraph chamada Shop4u
- Use blocos arredondados para os casos de uso
- Coloque os sistemas externos à direita
- Não detalhe telas, banco de dados ou implementação interna
- Retorne apenas o código Mermaid

13.3 Exemplo esperado em Mermaid



13.4 Como revisar este diagrama

Verifique se:

- o Cliente aparece como ator principal;
- os casos de uso estão dentro da fronteira do Shop4u;
- as funcionalidades principais estão representadas;
- o Serviço de IA está ligado ao caso de uso de recomendações;
- o Gateway de Pagamento está ligado à finalização da compra;
- o Serviço de Notificações está ligado à confirmação da compra;
- o diagrama evita detalhes técnicos de implementação.

13.5 Observação importante sobre o ator

No Mermaid, o Diagrama de Sequência possui suporte nativo para atores usando `actor`, por isso o Cliente pode aparecer como um bonequinho de palito nesse tipo de diagrama.

Já no Diagrama de Caso de Uso, estamos usando `flowchart`, porque o Mermaid não possui uma notação UML clássica completa para casos de uso. Por isso, o ator será representado como um bloco simples.

Isso não invalida o diagrama. Neste exercício, o objetivo é comunicar as funcionalidades acessadas pelo usuário de forma clara e organizada.

14. Diagrama UML de Sequência

14.1 O que este diagrama deve mostrar

O Diagrama de Sequência mostra a ordem das interações entre os participantes ao longo do tempo.

Neste caso, o fluxo escolhido é a finalização de compra no Shop4u.

A pergunta principal desse diagrama é:

Quem chama quem e em qual ordem?

Esse diagrama é útil para entender como Cliente, Mobile App, Backend API, Database, Gateway de Pagamento e Serviço de Notificações interagem durante o processo de compra.

14.2 Prompt recomendado

Gere um Diagrama UML de Sequência em Mermaid para o fluxo principal de compra do Shop4u.

Participantes:

- Cliente
- Mobile App
- Backend API
- Database
- Gateway de Pagamento
- Serviço de Notificações

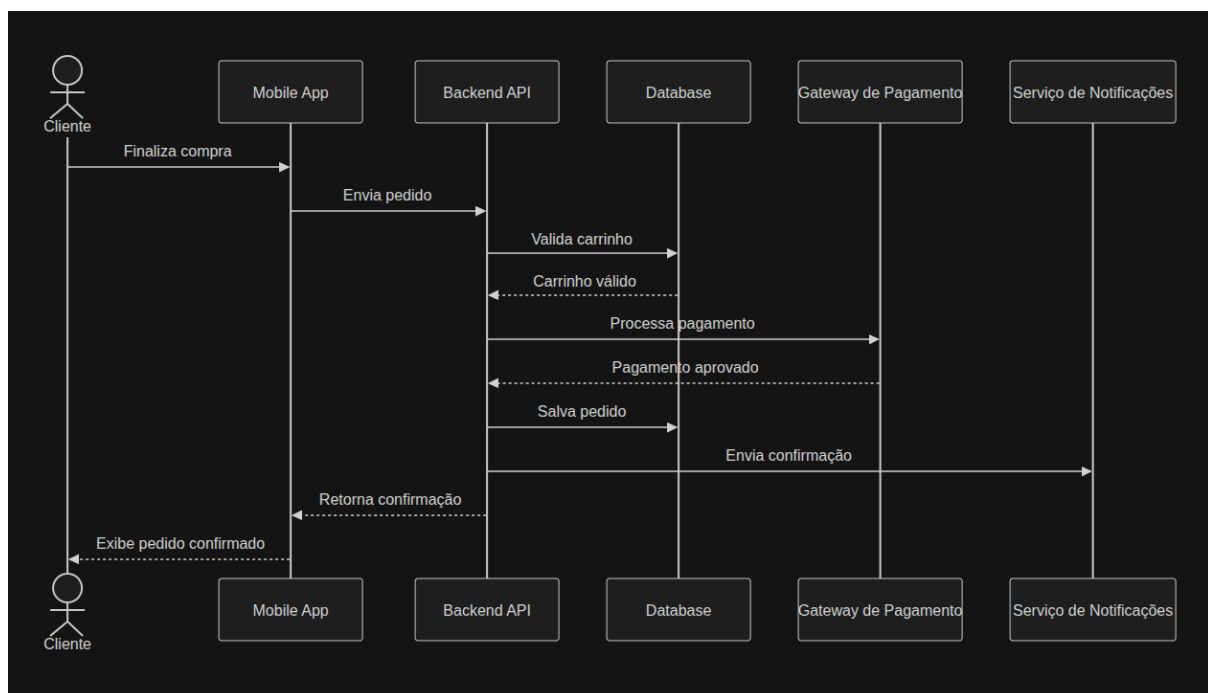
Fluxo:

1. Cliente finaliza a compra no Mobile App
2. Mobile App envia o pedido para o Backend API
3. Backend API valida o carrinho no Database
4. Database retorna os dados validados
5. Backend API chama o Gateway de Pagamento
6. Gateway de Pagamento retorna pagamento aprovado
7. Backend API salva o pedido no Database
8. Backend API chama o Serviço de Notificações
9. Backend API retorna a confirmação para o Mobile App
10. Mobile App exibe a confirmação para o Cliente

Regras:

- Use sequenceDiagram
- Represente Cliente como actor
- Use mensagens curtas
- Mostre a ordem das chamadas de cima para baixo
- Não detalhe implementação interna
- Retorne apenas o código Mermaid

14.3 Exemplo esperado em Mermaid



14.4 Como revisar este diagrama

Verifique se:

- os participantes estão claros;
 - o Cliente aparece como ator;
 - o fluxo começa com a ação do Cliente;
 - o Mobile App chama o Backend API;
 - o Backend API valida o carrinho antes de processar pagamento;
 - o Gateway de Pagamento retorna o resultado;
 - o pedido é salvo após a aprovação do pagamento;
 - a notificação é enviada após a criação do pedido;
 - a confirmação retorna para o Cliente.
-

15. Como usar os códigos Mermaid

Depois que a IA gerar o código Mermaid, copie o código e cole em uma ferramenta compatível.

Algumas opções:

- extensão Mermaid no editor usado em aula;
- Markdown Preview com suporte a Mermaid;
- GitHub, em arquivos Markdown compatíveis;
- Mermaid Live Editor;
- documentação do projeto em arquivos `.md`.

O bloco deve seguir este formato:

```
```mermaid
flowchart LR
 A --> B
```
```

Se o bloco não estiver marcado como `mermaid`, o editor pode mostrar apenas texto e não renderizar o diagrama.

16. Checklist geral de revisão

Depois de gerar os diagramas, revise o resultado com atenção.

Checklist geral

- O Cliente aparece no diagrama?

- O sistema principal está claro?
 - Os serviços externos foram identificados?
 - A IA aparece como serviço especializado?
 - O backend aparece como orquestrador no C4 Containers?
 - O banco de dados é acessado pelo backend?
 - O fluxo está em ordem lógica?
 - O diagrama está simples e compreensível?
 - Os textos dos blocos estão curtos?
 - As setas representam relações corretas?
 - O código Mermaid renderiza sem erro?
-

Checklist do C4 Contexto

- O Shop4u aparece como uma única caixa?
 - O Cliente está fora do sistema?
 - Serviço de IA, Gateway de Pagamento e Serviço de Notificações aparecem como sistemas externos?
 - O diagrama não detalha Mobile App, Backend API ou Database?
-

Checklist do C4 Containers

- O Mobile App aparece separado do Backend API?
 - O Backend API aparece como centro da orquestração?
 - O Database é acessado pelo Backend API?
 - Os serviços externos estão separados do sistema principal?
 - O diagrama evita detalhes internos como controllers, services, repositories, classes e métodos?
-

Checklist do Caso de Uso

- O Cliente aparece como ator principal?
 - As funcionalidades principais estão representadas?
 - Os casos de uso estão dentro da fronteira do Shop4u?
 - O diagrama mostra o que o usuário pode fazer?
 - O diagrama evita detalhes técnicos de implementação?
-

Checklist do Diagrama de Sequência

- Os participantes estão claros?
 - A ordem das interações faz sentido?
 - O Mobile App chama o Backend API?
 - O Backend API valida dados antes de processar pagamento?
 - O Backend API salva o pedido após o pagamento aprovado?
 - A confirmação retorna para o usuário?
-

17. Estratégias de refinamento para diagramas gerados por IA

A IA pode gerar diagramas incompletos, poluídos ou com relações incorretas. Isso é esperado.

Quando isso acontecer, ajuste o prompt ou peça uma revisão do Mermaid gerado.

Problema: o diagrama ficou com muito texto

Use este refinamento:

Simplifique este código Mermaid. Use nomes mais curtos nos blocos, reduza as descrições e mantenha apenas as informações essenciais.

Problema: o diagrama ficou técnico demais

Use este refinamento:

Remova detalhes internos de implementação. Não inclua controllers, services, repositories, classes, métodos ou arquivos. Mostre apenas os principais elementos do sistema.

Problema: faltou algum elemento

Use este refinamento:

Inclua o elemento [NOME DO ELEMENTO] no diagrama Mermaid e conecte esse elemento com [OUTRO ELEMENTO] usando a relação [RELAÇÃO].

Exemplo:

Inclua o Serviço de Notificações no diagrama Mermaid e conecte o Backend API a ele usando a relação "Envia confirmação".

Problema: as relações ficaram erradas

Use este refinamento:

Corrija as relações do Mermaid considerando:

- Cliente usa Mobile App
- Mobile App chama Backend API
- Backend API acessa Database
- Backend API consulta Serviço de IA
- Backend API chama Gateway de Pagamento
- Backend API chama Serviço de Notificações

Retorne apenas o código Mermaid corrigido.

Problema: o Mermaid não renderizou

Verifique se:

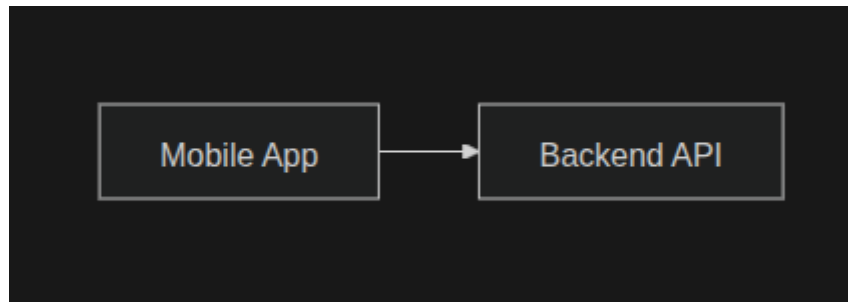
- o bloco começa com ````mermaid;`
- a sintaxe do tipo de diagrama está correta;
- você copiou apenas o código Mermaid;
- não há explicações misturadas dentro do bloco;
- os parênteses, colchetes e chaves estão fechados corretamente;
- os nomes internos dos nós não usam espaços.

Exemplo incorreto:

```
flowchart LR
  Mobile App --> Backend API
```

Exemplo corrigido:

```
flowchart LR
  mobileApp[Mobile App] --> backendApi[Backend API]
```



No Mermaid, o identificador interno deve ser simples. O nome visual pode ficar entre colchetes.

18. Entrega esperada

Cada equipe deve entregar:

1. Diagrama C4 de Contexto em Mermaid.
2. Diagrama C4 de Containers em Mermaid.
3. Diagrama UML de Caso de Uso em Mermaid.
4. Diagrama UML de Sequência em Mermaid.
5. Breve explicação de cada diagrama.

A explicação de cada diagrama deve responder:

- Qual é o objetivo do diagrama?
 - Quais elementos aparecem nele?
 - Que entendimento ele ajuda a comunicar?
-

19. Critérios de qualidade da entrega

A entrega será considerada melhor quando os diagramas estiverem:

- coerentes com o estudo de caso;
- simples de entender;
- tecnicamente consistentes;
- sem excesso de elementos;
- com relações corretas;
- com código Mermaid funcional;
- acompanhados de explicação breve e objetiva.

O objetivo da atividade não é criar o diagrama mais complexo possível. O objetivo é representar bem o sistema, usando a IA como apoio e mantendo a revisão crítica da equipe.

